COLLEGE EVENT MANAGEMENT SYSTEM WITH CI/CD

Complete Academic Project Architecture & Step-by-Step Beginner Execution Guide

Subject: DevOps & Cloud Computing	**Project Scope:** Minimal & Deterministic CI/CD
Tech Stack: Python, Flask, SQLite, PyTest, Docker, GitHub Actions	**Target Audience:** Absolute Beginners, College Students & Examiners

1. Project Overview & Academic Foundation

Abstract: Software engineering education frequently emphasizes writing application code but overlooks how software is tested, packaged, and continuously deployed in production environments. This project presents a streamlined **College Event Management System** written in Python and Flask with an embedded SQLite database, coupled with a production-grade **DevOps Continuous Integration & Continuous Deployment (CI/CD) Pipeline** managed by GitHub Actions and Docker. The primary mission of this project is to illustrate the end-to-end software delivery lifecycle in a clear, minimal, and fully demonstrable manner.

Problem Statement: Traditional manual software delivery suffers from configuration drift, human deployment error, and the classic 'it works on my machine' dilemma. In academic environments, students often struggle to deploy multi-tiered enterprise frameworks. This project solves both issues by packaging essential business logic into an automated quality-gated delivery pipeline.

Core Design Principle: *Maximum simplicity in application logic so that 100% of the demonstration focus is on the DevOps toolchain, automated unit testing, container isolation, and cloud automation.*

2. System Architecture & Database Design

The project follows a clean three-tier architecture implemented with zero heavyweight dependencies:

- **Presentation Layer:** Single-page responsive interface (templates/index.html, static/style.css) providing an event creation form and visual event cards.
- **Application Layer:** Python Flask server (app.py) implementing clean RESTful HTTP endpoints for listing (GET /), adding (POST /add), and deleting (POST /delete/<id>) events.
- **Persistence Layer:** Single-file embedded relational SQLite database (events.db) managed with parameterized SQL queries via Python's native sqlite3 library.

Relational Database Schema (Table: events):

Column	Data Type	Constraint	Description
id	INTEGER	PRIMARY KEY AUTOINCREMENT	Unique event identifier
name	TEXT	NOT NULL	Title/Name of the college event
date	TEXT	NOT NULL	Scheduled date (YYYY-MM-DD)
venue	TEXT	NOT NULL	Campus auditorium, lab, or hall
description	TEXT	NOT NULL	Summary details and activity agenda

3. The DevOps Lifecycle & CI/CD Explained

What is DevOps? DevOps is the combination of cultural philosophies, practices, and tools that increases an organization's ability to deliver applications at high velocity. In this project, DevOps bridges the gap between writing Python code and delivering a running container.

Phase	How It Is Implemented in This Project
1. PLAN	Define requirements: lightweight event scheduling, listing, and deletion with zero authentication overhead.
2. CODE	Develop Flask routes in app.py and styling in templates/index.html and static/style.css.
3. BUILD	Resolve dependencies via pip install and assemble the container using Dockerfile.
4. TEST	Execute automated unit test suite with pytest using an isolated temporary test database.
5. RELEASE	Commit verified code to the GitHub version control repository on the main branch.
6. DEPLOY	Trigger cloud container deployment automatically via GitHub Actions deployment hook.
7. OPERATE	Run the containerized application on port 5000 serving live HTTP user requests.
8. MONITOR	Inspect GitHub Actions build logs, container health status, and Flask access logs.
9. FEEDBACK	Catch failed test runs or broken routes immediately; refine code and re-push through the pipeline.

4. Step-by-Step Beginner Execution Guide

*This section is designed so that **anyone with zero prior DevOps experience** can successfully set up, run, test, and demonstrate this project from scratch.*

Phase A: Local Setup & Prerequisites

Ensure you have installed:

1. **Python 3.10+** (from python.org) — *Crucial: Check the box 'Add python.exe to PATH' during installation.*
2. **Git** (from git-scm.com)
3. **Visual Studio Code** (from code.visualstudio.com)
4. **Docker Desktop** (Optional, for container testing from docker.com)

Step 1: Open the Project in VS Code

Open VS Code, press **Ctrl +** ` (backtick) or go to **Terminal > New Terminal**. Navigate to your project folder:

```
cd c:\Projects\DevOPS-Eshwar
```

Step 2: Create & Activate Virtual Environment

A virtual environment prevents conflicts with other Python projects on your computer.

```
On Windows PowerShell:
python -m venv venv
.\venv\Scripts\Activate.ps1

Note: If PowerShell shows a script execution error, run:
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass
and rerun the Activate command.
```

Step 3: Install Required Dependencies

Install Flask and PyTest using the provided requirements file:

```
pip install -r requirements.txt
```

Step 4: Run the Application Locally

Launch the Flask development server:

```
python app.py
```

The terminal will display:

```
* Serving Flask app 'app'
* Running on http://127.0.0.1:5000
* Running on http://0.0.0.0:5000
```

Open your web browser (Chrome, Edge, Firefox) and navigate to **http://127.0.0.1:5000**. You will see the College Event Management System pre-loaded with 3 default sample events!

Step 5: Demonstrate Adding and Deleting Events

- **Add an Event:** Type 'National Hackathon' into Event Name, select tomorrow's date, enter 'Computer Center' as Venue, type '24-hour coding challenge' into Description, and click **[Add Event]**. The new event immediately appears on top of the list.
- **Delete an Event:** Click the red **[Delete]** button beside any event card. Confirm the browser prompt; the event is instantly erased from SQLite and the screen refreshes.

Press **Ctrl + C** in your VS Code terminal to stop the local Flask server when done.

5. Automated Testing & Containerization

Step 6: Run Automated Unit Tests with PyTest

DevOps relies heavily on automated test suites. Execute the PyTest command:

```
pytest -v
```

Expected Output:

```
============================= test session starts ==============================
collected 4 items

tests/test_app.py::test_app_starts PASSED [ 25%]
tests/test_app.py::test_home_page_returns_200 PASSED [ 50%]
tests/test_app.py::test_add_event PASSED [ 75%]
tests/test_app.py::test_delete_event PASSED [100%]

============================== 4 passed in 0.63s ===============================
```

Examiner Tip: *Explain that tests/test_app.py uses a dynamic pytest fixture to spin up a temporary database in memory/temp storage. Testing* ***never*** *deletes or overwrites the actual student events database!*

Step 7: Build & Run with Docker

Docker packages the code, Python runtime, libraries, and web server into a lightweight, standalone container.

1. **Build the container image:**

```
docker build -t college-events .
```

2. **Run the container on port 5000:**

```
docker run -p 5000:5000 college-events
```

3. **Access in browser:** Visit **http://localhost:5000**. The application runs identically inside the isolated container.

6. Version Control & GitHub Actions CI/CD Pipeline

Follow these exact commands to push the project to GitHub and trigger the pipeline:

```
git init
git add .
git commit -m "Initial commit: College Event Management with CI/CD"
git branch -M main
git remote add origin https://github.com/YOUR_USERNAME/YOUR_REPO_NAME.git
git push -u origin main
```

How to View the Live Automated Pipeline on GitHub:

1. Open your repository on GitHub in any web browser.
2. Click on the **Actions** tab near the top of the repository.
3. Click on the active workflow run labeled **CI/CD Pipeline**.
4. Observe the automated sequence of jobs executing in real-time:

```
    [✓] 1. Checkout Repository
    [✓] 2. Set up Python 3.11 Environment
    [✓] 3. Install Dependencies from requirements.txt
    [✓] 4. Run Automated PyTest Suite (4 tests pass)
    [✓] 5. Build Docker Image (college-events:latest)
    [✓] 6. Continuous Deployment (Webhook execution)
```

What happens if a test fails? *If any developer pushes broken code (for example, breaking the delete route), PyTest exits with code 1. GitHub Actions immediately marks the pipeline with a red cross, blocks the Docker build, and stops deployment.*

7. Complete Live Demonstration Script (For Viva & Presentation)

Use this structured script during your evaluation or project viva:

Step & Action	Command / URL	What to Say to the Examiner
1. Start App Locally	python app.py *http://127.0.0.1:5000*	"Here the Flask web application starts on port 5000. It connects to SQLite and automatically initializes the database with 3 sample events if the table does not exist."
2. Add New Event	Fill form in browser & submit	"I am submitting an event form. The POST request reaches /add, inserts the record into SQLite, and re-renders the list without requiring page reloads or complex state."
3. Delete Event	Click **[Delete]** button	"Clicking Delete triggers /delete/<id>. The record is removed from the SQLite database with SQL parameter substitution to prevent SQL injection."
4. Run PyTest	pytest -v	"DevOps requires automated verification before deployment. We run pytest, which executes 4 unit tests against an isolated temporary database in under 1 second."
5. Docker Packaging	docker build -t college-events . docker run -p 5000:5000 college-events	"To solve 'it works on my machine', Docker encapsulates Python 3.11-slim, requirements, and app code into a portable container image accessible at localhost:5000."
6. Show GitHub Actions	GitHub Repo > **Actions** tab	"Whenever code is pushed to GitHub, our CI/CD pipeline triggers automatically on Ubuntu runners: checking out code, installing dependencies, testing, and building the container."

8. Viva Voce Questions & Answers (Frequently Asked by Examiners)

Q1: What is the primary purpose of this project?

Answer: The primary purpose is to demonstrate the practical DevOps lifecycle and CI/CD automation using a minimalist, functional Flask web application, Docker containerization, and GitHub Actions.

Q2: What is the difference between Continuous Integration (CI) and Continuous Deployment (CD)?

Answer: Continuous Integration (CI) automatically builds and tests code on every commit to catch bugs early. Continuous Deployment (CD) automatically releases that validated code to production servers without manual human intervention.

Q3: Why did you choose SQLite over MySQL or PostgreSQL?

Answer: SQLite requires zero external server setup, runs natively within Python's built-in sqlite3 library, and enables fast, lightweight testing with zero installation friction, perfectly aligning with a minimal academic demonstration.

Q4: How does your test suite avoid modifying real application data?

Answer: In tests/test_app.py, a PyTest fixture generates an isolated temporary database file (using tempfile.mkstemp). The application routes point to this temporary database during testing, and the file is deleted upon test completion.

Q5: What is the significance of the Dockerfile?

Answer: The Dockerfile defines the blueprint to package Python 3.11-slim, all dependencies in requirements.txt, and application source code into an immutable container image that runs identically on any OS.

Q6: How does GitHub Actions prevent broken code from being deployed?

Answer: GitHub Actions executes sequentially. If any test in PyTest fails, the workflow immediately exits with a non-zero exit code. Subsequent steps (Docker build and deployment) are automatically skipped.

9. Conclusion

The **College Event Management System with CI/CD** represents an optimal academic balance: it eliminates unnecessary software complexity while rigorously demonstrating real-world DevOps practices. By mastering this project, a student can confidently explain version control, automated testing, containerization, and cloud delivery pipelines to any technical evaluator.